

Modelos Generativos Profundos

Clase 10: Modelos autorregresivos y LLMs (parte VI)

Fernando Fêtis Riquelme

Otoño, 2026

Facultad de Ciencias Físicas y Matemáticas
Universidad de Chile

Estrategias de decodificación

Inferencia eficiente

Prompting

Agentes

Estrategias de decodificación

Recuerdo de modelos de lenguaje

Un modelo de lenguaje autorregresivo modela distribuciones condicionales de la forma

$$p_{\theta}(w_{t+1}|w_1, \dots, w_t) \sim \text{Categ\acute{o}rica}(\mathcal{V}; \lambda_{\theta}(w_1, \dots, w_t)),$$

donde $\lambda_{\theta}(w_1, \dots, w_t) \in \Delta^{|\mathcal{V}|}$ es un vector de probabilidades sobre $\mathcal{V}$, el cual depende del contexto.

Una red neuronal (e.g. RNN o GPT) recibe el contexto y entrega un vector de logits $l \in \mathbb{R}^{|\mathcal{V}|}$ que permite definir $\lambda_{\theta} = \text{softmax}(l)$.

La generaci3n de texto utiliza iterativamente las distribuciones $w_{t+1} \sim \text{Categ\acute{o}rica}(\mathcal{V}; \lambda_{\theta}(w_1, \dots, w_t))$ utilizando distintas **t3cnicas de decodificaci3n**.

Recuerdo de temperatura

Una heurística usual consiste en utilizar la idea de **temperatura**, donde los logits $l \in \mathbb{R}^{|\mathcal{V}|}$ son reescalados según una constante $\tau > 0$ antes de softmax:

$$\lambda_{\theta}^{(\tau)} = \text{softmax} \left(\frac{l}{\tau} \right) \in \Delta^{|\mathcal{V}|}$$

- $\tau \rightarrow 0$ equivale a greedy decoding.
- $\tau > 1$ permite una mayor diversidad.
- $\tau < 1$ puede mejorar coherencia a costa de diversidad.

Recuerdo de top- K sampling

Otra técnica usual es **top- K sampling**, donde se restringe la distribución a asignar masa solo a los K tokens más probables, $\mathcal{I} = \{k_1, \dots, k_K\}$:

$$\tilde{\lambda}_{\theta,k} = \begin{cases} \frac{\lambda_{\theta,k}}{\sum_{i \in \mathcal{I}} \lambda_{\theta,i}} & \text{si } k \in \mathcal{I} \\ 0 & \text{si } k \notin \mathcal{I} \end{cases}$$

Con esto, el decoding se realiza considerando el vector de probabilidad $\tilde{\lambda}_{\theta} \in \Delta^{|\mathcal{V}|}$ en lugar de $\lambda_{\theta} \in \Delta^{|\mathcal{V}|}$.

Sin embargo, en este método K es fijo y no se adapta a la forma de la distribución $p_{\theta}(w_{t+1}|w_1, \dots, w_t)$.

Top- p sampling

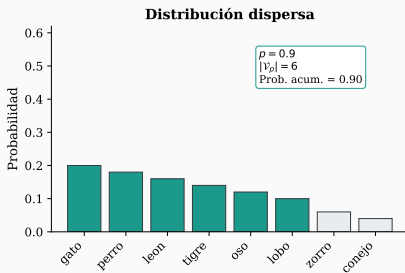
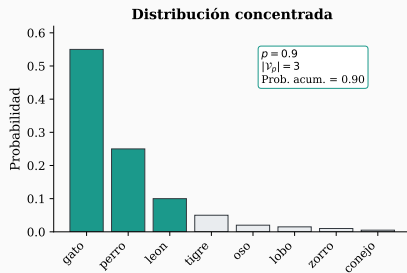
Una versión dinámica de top- K sampling es **nucleus sampling**, donde se selecciona el subconjunto más pequeño de tokens cuya masa acumulada es al menos $p \in (0, 1]$.

Si $\lambda_{\sigma(1)} \geq \lambda_{\sigma(2)} \geq \dots \geq \lambda_{\sigma(|\mathcal{V}|)}$ son las componentes de λ_θ ordenadas de mayor a menor, nucleus sampling distribuye la masa de λ_θ sobre el conjunto $\mathcal{V}_p = \{a_{\sigma(1)}, \dots, a_{\sigma(K_p)}\}$, donde

$$K_p = \min \left\{ K \in \{1, \dots, |\mathcal{V}|\} : \sum_{i=1}^K \lambda_{\sigma(i)} \geq p \right\}$$

A diferencia de top- K , el tamaño del conjunto $\mathcal{V}_p \subset \mathcal{V}$ varía dinámicamente.

Top- p sampling



Beam search

Las técnicas anteriores solo modifican la distribución del próximo token, pero no consideran la secuencia completa.

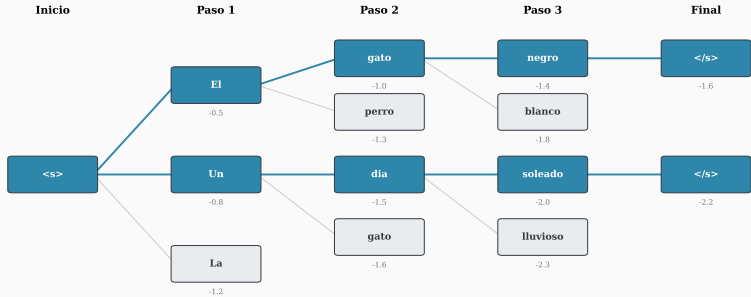
La técnica de **beam search** mantiene las $B \geq 1$ secuencias más probables en cada paso de generación y les asigna un **score**:

$$\text{score}(w_1, \dots, w_t) := \frac{1}{t^\alpha} \log p_\theta(w_1, \dots, w_t),$$

donde $\alpha \in [0, 1]$ es un hiperparámetro de normalización que evita favorecer secuencias cortas.

En cada paso, se generan todas las continuaciones posibles de las B secuencias actuales, se calculan sus scores, y se retienen solo las B secuencias con mayor score para el siguiente paso.

Beam search



Inferencia eficiente

La inferencia de LLMs presenta varios desafíos:

- **Latencia:** a diferencia del entrenamiento, la generación token a token es inherentemente secuencial.
- **Memoria:** modelos grandes no caben en una sola GPU.
- **Throughput:** servir muchos usuarios simultáneamente requiere optimización.

Existen técnicas para optimizar a nivel de **arquitectura** (MQA, GQA), **cómputo** (KV caching) y **compresión** (cuantización, pruning y/o destilación).

KV cache

El mecanismo de atención define proyecciones $k_t = W_K x_t \in \mathbb{R}^P$ y $v_t = W_V x_t \in \mathbb{R}^P$ con $W_K, W_V \in \mathcal{M}_{P,D}(\mathbb{R})$.

Sin embargo, para generar el token $w_{t+1} \in \mathcal{V}$ se necesitan las proyecciones de todos los tokens previos, lo que implica recalcular $k_i, v_i \in \mathbb{R}^P$ para cada $i \in \{1, \dots, t\}$ en cada paso.

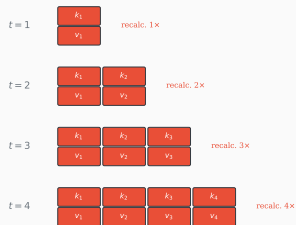
La técnica de **KV caching** almacena estas proyecciones para reutilizarlas en la generación:

$$K_{1:t} = \begin{pmatrix} k_1^\top \\ \vdots \\ k_t^\top \end{pmatrix} \in \mathcal{M}_{t,P}(\mathbb{R}), \quad V_{1:t} = \begin{pmatrix} v_1^\top \\ \vdots \\ v_t^\top \end{pmatrix} \in \mathcal{M}_{t,P}(\mathbb{R})$$

Esto permite reducir drásticamente el tiempo de cómputo en cada paso, a costa de un overhead de memoria que crece linealmente con la longitud de la secuencia.

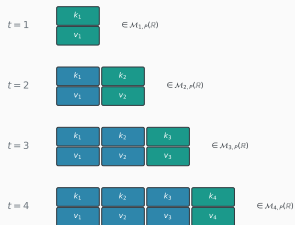
KV cache

Sin KV cache



Recalculado

Con KV cache



En cache
Nuevo

Considerando que MHA utiliza $H \in \mathbb{N}$ cabezas, cada una con matrices de proyección individuales, el costo de memoria para almacenar el KV cache puede ser muy alto.

Existen dos alternativas que reducen este costo:

- **Multi-Query Attention (MQA):** todas las cabezas comparten un único par $W_K, W_V \in \mathcal{M}_{P,D}(\mathbb{R})$. Cada cabeza mantiene su $W_Q^{(h)}$.
- **Grouped-Query Attention (GQA):** las H cabezas se agrupan en G grupos, cada grupo comparte un par $W_K^{(g)}, W_V^{(g)} \in \mathcal{M}_{P,D}(\mathbb{R})$.

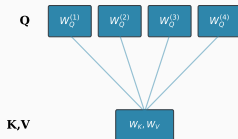
Estas variantes de MHA reducen el costo de memoria del KV cache y, en la práctica, no degradan significativamente la calidad del modelo.

MHA, MQA y GQA

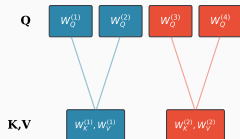
Multi-Head (MHA)



Multi-Query (MQA)



Grouped-Query (GQA)



Cuantización

La técnica de **cuantización** reduce la precisión numérica de los distintos parámetros de un modelo neuronal:

- FP32 $\rightarrow$ FP16: 2 $\times$ reducción de memoria.
- FP32 $\rightarrow$ INT8: 4 $\times$ reducción.
- FP32 $\rightarrow$ INT4: 8 $\times$ reducción.

Existen dos enfoques principales:

- **Post-Training Quantization (PTQ):** cuantiza un modelo ya entrenado. Dos métodos populares son GPTQ y AWQ.
- **Quantization-Aware Training (QAT):** simula la cuantización durante el entrenamiento para que el modelo aprenda a ser robusto a ella. Es de mejor calidad pero también mayor costo.

Pruning

El **pruning** de una red neuronal consiste en eliminar pesos o estructuras del modelo que contribuyen poco a la salida:

- **Unstructured pruning:** elimina pesos individuales por magnitud u otros criterios. Produce matrices sparse difíciles de acelerar en hardware estándar.
- **Structured pruning:** elimina estructuras completas (neuronas, cabezas de atención, capas). Más fácil de acelerar en hardware.

Para esto, lo usual es entrenar un modelo completo, identificar pesos poco importantes, eliminarlos, y (opcionalmente) hacer fine-tuning para recuperar rendimiento.

Knowledge distillation

La **destilación** de un modelo consiste en entrenar un modelo **student** p_{θ_S} (pequeño) para imitar a un modelo **teacher** p_{θ_T} (grande) sobre un dataset $\mathcal{D} = \{S^1, \dots, S^N\} \subset \mathcal{V}^*$.

Para cada posición t de una secuencia, sean $l_T, l_S \in \mathbb{R}^{|\mathcal{V}|}$ los logits temperados del teacher y student:

$$\lambda_T^{(\tau)} = \text{softmax}(l_T/\tau) \in \Delta^{|\mathcal{V}|}, \quad \lambda_S^{(\tau)} = \text{softmax}(l_S/\tau) \in \Delta^{|\mathcal{V}|}$$

La función de pérdida de destilación combina hard labels (token real w_{t+1}) y soft labels (distribución del teacher):

$$\mathcal{L}_t^{\text{KD}}(\theta_S) = \underbrace{\alpha \cdot \text{CE}(w_{t+1}, \lambda_S^{(1)})}_{\text{hard labels}} + (1 - \alpha) \cdot \tau^2 \cdot \underbrace{\text{KL}(\lambda_T^{(\tau)} \parallel \lambda_S^{(\tau)})}_{\text{soft labels}}$$

con $\alpha \in [0, 1]$ un hiperparámetro. Luego, $\mathcal{L}_t^{\text{KD}}(\theta_S)$ se promedia sobre todas las posiciones y secuencias de $\mathcal{D}$.

Knowledge distillation

El uso de una temperatura $\tau > 0$ suaviza las distribuciones, revelando las probabilidades relativas de tokens no dominantes (**dark knowledge**). El factor τ^2 corrige la magnitud de los gradientes al escalar los logits.

Existen otras variantes de destilación:

- **Destilación de capas intermedias:** alinear representaciones internas del modelo.
- **Destilación con datos sintéticos:** el modelo teacher genera secuencias de alta calidad y el modelo student se entrena sobre ellas.

Prompting

In-Context Learning

in-context learning (ICL) es la capacidad de los LLMs de realizar tareas especificadas únicamente mediante el prompt, sin actualizar sus parámetros.

Esta capacidad emerge del pre-training sobre texto diverso, donde el modelo aprende a reconocer patrones en el contexto y a continuar coherentemente.

- **Zero-shot:** solo se describe la tarea, sin ejemplos.
- **Few-shot:** se entregan k ejemplos resueltos antes de la consulta.
- **Instruction-based:** la tarea se describe con instrucciones simples en lenguaje natural.

GPT-3 demostró que la capacidad ICL mejora con la escala del modelo.

In-Context Learning

Zero-shot:

Clasifica el sentimiento como positivo o negativo.

Texto: La película fue increíble. Sentimiento:

Few-shot:

Clasifica el sentimiento como positivo o negativo.

Texto: Me encantó el libro. Sentimiento: positivo

Texto: El servicio fue terrible. Sentimiento: negativo

Texto: La película fue increíble. Sentimiento:

CoT prompting induce al modelo a generar pasos de razonamiento intermedios antes de la respuesta final. Existen dos variantes principales:

- **Few-shot CoT:** incluye ejemplos con razonamiento explícito.
- **Zero-shot CoT:** agrega “think step by step” al prompt.

Si bien esta es una técnica trivial de utilizar, hoy en día existen resultados teóricos acerca de su funcionamiento y la clase de problemas que permite resolver.

Chain of Thought

Zero-shot CoT:

P: Si tengo 3 manzanas y compro 2 bolsas con 4 manzanas cada una, ¿cuántas tengo?
Pensemos paso a paso.

Respuesta del modelo:

1. Empiezo con 3 manzanas.
2. Cada bolsa tiene 4 manzanas.
3. Compro 2 bolsas: $2 \times 4 = 8$ manzanas.
4. Total: $3 + 8 = 11$ manzanas.

Self-refinement

El **self-refinement** permite al modelo mejorar iterativamente su respuesta:

Generate → Critique → Refine → ...

1. El modelo genera una respuesta inicial.
2. El mismo modelo (con un prompt distinto) critica la respuesta, identificando errores.
3. El modelo refina la respuesta incorporando la crítica.

Es efectivo cuando el modelo puede juzgar la calidad de una respuesta mejor de lo que puede generarla óptimamente en un solo intento.

No requiere herramientas externas ni memoria, sino que es una técnica puramente de prompting.

Prompt tuning

El **prompt tuning** adapta un LM congelado p_θ a una tarea específica agregando k vectores continuos entrenables a la entrada.

Dados los embeddings de entrada $x_1, \dots, x_T \in \mathbb{R}^D$ y vectores entrenables $p_1, \dots, p_k \in \mathbb{R}^D$ (agrupados en $P \in \mathcal{M}_{k,D}(\mathbb{R})$):

entrada al modelo: $(p_1, \dots, p_k, x_1, \dots, x_T)$

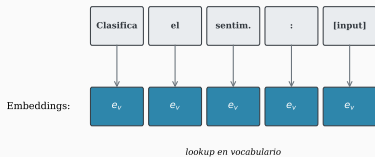
Se optimiza P manteniendo θ congelado:

$$\min_P \frac{1}{|\mathcal{D}_{\text{task}}|} \sum_{(x,y) \in \mathcal{D}_{\text{task}}} \mathcal{L}(p_\theta(y|P, x), y)$$

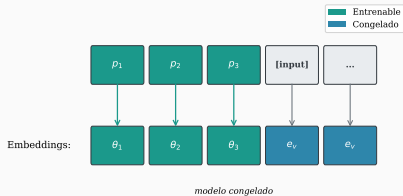
Esta técnica corresponde a un caso de PEFT, donde el prompt incluye un conjunto de parámetros continuos optimizados para la tarea.

Prompt tuning

Hard prompting



Soft prompting



Agentes

Agentes basados en LLMs

Un **agente** basado en un LLM usa el modelo como controlador central para razonar, planificar y ejecutar acciones en un entorno.

Componentes:

- **LLM** (p_θ): procesa información, razona y decide acciones.
- **Herramientas** ($\mathcal{T} = \{f_1, \dots, f_M\}$): funciones externas invocables (APIs, búsqueda web, intérprete de código).
- **Memoria** ($\mathcal{M}$): almacena historial de interacciones y contexto relevante más allá de la ventana de contexto.
- **Planificación**: descomposición de tareas complejas en sub-tareas ejecutables.

En cada paso, el agente observa el estado actual, razona sobre qué acción tomar, ejecuta la acción y observa el resultado.

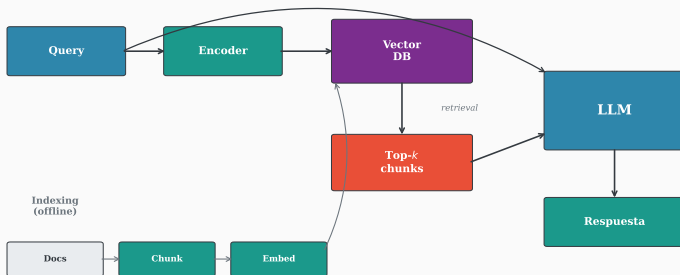
Retrieval-Augmented Generation

RAG aumenta el LLM con conocimiento externo recuperado de una base de datos vectorial. Dado un query x :

1. **Indexing (offline)**: los documentos se dividen en fragmentos $\{d_1, \dots, d_N\}$ y se codifican como embeddings $e_i = \text{enc}(d_i) \in \mathbb{R}^D$ en una base vectorial.
2. **Retrieval**: se codifica el query $e_x = \text{enc}(x)$ y se recuperan los k fragmentos más cercanos: $\{d_{i_1}, \dots, d_{i_k}\}$.
3. **Generation**: el LLM genera condicionado en el query y los fragmentos: $p_\theta(\text{respuesta}|x, d_{i_1}, \dots, d_{i_k})$.

RAG reduce alucinaciones al fundamentar respuestas en documentos, permite acceso a conocimiento actualizado sin re-entrenar, y permite citar fuentes.

Retrieval-Augmented Generation



Reflexion

Reflexion extiende el self-refinement con **memoria persistente** entre episodios, convirtiéndolo en un comportamiento agéntico:

1. El agente intenta resolver la tarea.
2. Se evalúa el resultado mediante una señal externa (tests, feedback, verificador).
3. Si falla, el modelo genera una **reflexión verbal** sobre el error cometido.
4. La reflexión se almacena en memoria $\mathcal{M}$ y se incluye en el contexto de futuros intentos.

A diferencia del self-refinement (un solo episodio, sin memoria), Reflexion opera a través de múltiples episodios, acumulando experiencia. Esto permite “aprender de errores” sin actualizar los pesos del modelo.

Tool use y function calling

Los LLMs pueden invocar **herramientas externas** $f_i \in \mathcal{T}$ para extender sus capacidades. El mecanismo de **function calling**:

1. Se definen herramientas disponibles con nombre, descripción y esquema de parámetros (provisto en el prompt del sistema).
2. El modelo decide si invocar una función y genera los argumentos en formato JSON.
3. El sistema ejecuta la función y retorna el resultado al modelo.

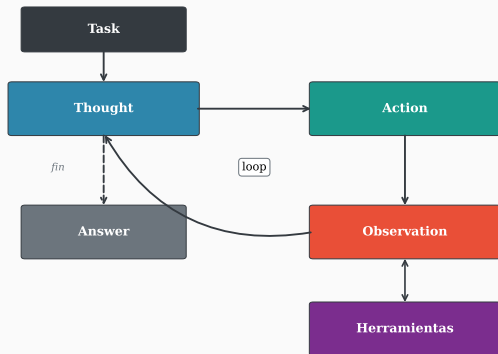
Toolformer mostró que un LLM puede aprender autónomamente cuándo y cómo invocar herramientas, insertando llamadas a APIs en el texto generado durante el entrenamiento.

ReAct (Reasoning + Acting) intercala razonamiento explícito y acciones en un loop:

1. **Thought:** el modelo razona (en texto) sobre qué hacer.
2. **Action:** ejecuta una acción $a_t = f_i(\text{args})$ invocando una herramienta $f_i \in \mathcal{T}$.
3. **Observation:** recibe el resultado o_t de la acción.

El ciclo se repite: el modelo genera el siguiente thought condicionado en todo el historial $(q, a_1, o_1, \dots, a_t, o_t)$ hasta completar la tarea.

ReAct combina las ventajas de CoT (razonamiento trazable) con la capacidad de interactuar con el entorno, superando las limitaciones de ambos enfoques por separado.



Question: ¿Cuál es la población de la capital de Francia?

Thought: Necesito encontrar la capital de Francia.

Action: search("capital de Francia")

Observation: La capital de Francia es París.

Thought: Ahora necesito la población de París.

Action: search("población de París")

Observation: París tiene aproximadamente 2.1 millones.

Thought: Tengo la respuesta.

Answer: La población de París es aprox. 2.1 millones.

Model Context Protocol

El **MCP** es un estándar abierto para conectar LLMs con herramientas y datos de manera uniforme. Define tres roles:

- **Host:** la aplicación del usuario (e.g. VS Code, Claude Desktop). Crea y gestiona clientes MCP.
- **Client:** componente dentro del host que establece una conexión 1:1 con un servidor MCP.
- **Server:** proceso que expone capacidades al LLM:
 - **Tools:** funciones invocables (e.g. `search_issues(repo, query)`).
 - **Resources:** datos consultables (e.g. contenido de archivos).
 - **Prompts:** plantillas sugeridas para tareas comunes.

MCP estandariza la integración ya que un servidor puede ser consumido por cualquier host compatible (similar a como USB estandarizó la conexión de periféricos).

En la próxima clase:

- Introducción a las GANs.

Modelos Generativos Profundos

Clase 10: Modelos autorregresivos y LLMs (parte VI)